

urdu-slm

A from-scratch Urdu language model

Deep Dive — exhaustive technical walkthrough

Project by Salman Adnan

Document generated 2026-07-07

What this document is. A beginner-safe, exhaustive explanation of every component in **urdu-slm**: the data pipeline, the tokenizer, the transformer model, the training loop, evaluation, and the real results this project has produced, including the newly completed GPU training run. Every number here is read from the project's own source code, configs, and logged output — nothing is invented. Where something is an inference rather than a directly measured fact, it is labeled as such.

Contents

1	What this project is	2
1.1	Why Urdu specifically	2
1.2	The pipeline end to end	2
2	The data pipeline (pipeline/)	3
2.1	Sources (pipeline/sources.py)	3
2.2	Normalization (pipeline/normalize.py)	3
2.3	Language identification without a heavyweight model	3
2.4	Deduplication (pipeline/dedup.py)	4
2.4.1	The performance rewrite	4
2.5	Corpus build stages (pipeline/build_corpus.py)	5
2.6	The real corpus that was built (measured numbers)	5
3	The tokenizer	6
3.1	Measured compression against GPT-2's tokenizer	6
4	The model (model/)	7
4.1	Architecture overview	7
4.2	RMSNorm	7
4.3	Rotary Position Embeddings (RoPE)	8
4.4	Attention, and grouped-query capability	8
4.5	SwiGLU MLP	8
4.6	Tied embeddings	9
4.7	Weight initialization	9
4.8	Size presets	9
5	Training (train.py)	10
5.1	The token loader	10

5.2	Learning-rate schedule	10
5.3	Precision, gradient accumulation, and the training step	10
5.4	Token-count scheduling	10
5.5	Checkpointing, resume, and DDP	10
6	Training runs actually executed	12
6.1	Phase 1: the tiny CPU proof run	12
6.2	Phase 2: the base-model GPU run — completed	13
6.2.1	Token budget reasoning	14
6.2.2	Base-model evaluation (measured after training)	14
7	Evaluation harness (eval/)	16
8	Tests (tests/)	16
9	Project decisions worth knowing about	17
9.1	Lessons the author recorded	17
9.2	What the author says they would do differently	17
9.3	Licensing	17
10	Current status, honestly	18

1 What this project is

`urdu-slm` builds a small language model for Urdu entirely from scratch: no pretrained tokenizer, no modeling code borrowed from the Hugging Face `transformers` library, and a training pipeline built by hand from raw public text to a trained transformer checkpoint. It exists to demonstrate the parts of “train a GPT” that most tutorials skip because they default to English: how do you clean and deduplicate a large noisy web/wiki corpus, how do you build a tokenizer that does not fall apart on a non-Latin script, and how do you run and reason about a training budget honestly when your corpus is small?

What is a language model?

A language model is a program that has learned, from a lot of example text, to predict what word (or word-fragment) comes next given what came before. Chained one prediction at a time, that next-token predictor can generate whole sentences. “Decoder-only transformer” (the architecture used here) is the specific neural network design behind most modern text-generation models, including GPT-style systems.

1.1 Why Urdu specifically

Urdu is written right-to-left in the Perso-Arabic script and has rich morphology (words change form to carry grammatical meaning that English expresses with separate words). Off-the-shelf tools built for English — like GPT-2’s tokenizer — handle this badly: Section 3 shows this project’s own measurement that GPT-2’s tokenizer needs **5.23x** more tokens than a purpose-built Urdu tokenizer for the same text. That is not a minor inefficiency; it means more compute and money to train and run a model on the same amount of Urdu content.

1.2 The pipeline end to end

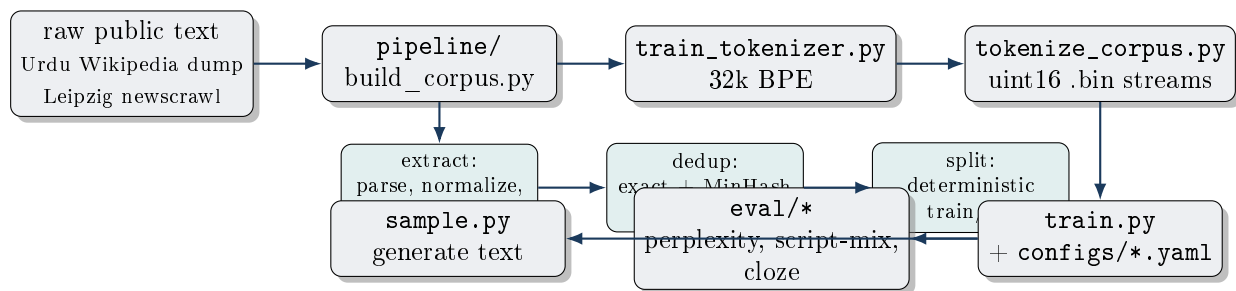


Figure 1: Corpus-to-model pipeline. Each labeled box is a runnable script or module in the repository.

Each stage writes its output to disk before the next begins, and the corpus builder is resumable: a `.stage.done` marker file records that a stage finished, so re-running the command skips completed work unless `-force` is passed (`pipeline/common.py`, class `Stage`).

2 The data pipeline (pipeline/)

2.1 Sources (pipeline/sources.py)

Three source readers are wired up, chosen because all are downloadable without authentication (a project decision documented in `docs/DATA_LICENSES.md`):

- **Urdu Wikipedia dump** — a MediaWiki XML dump, streamed with `xml.etree.ElementTree.iterparse` so the whole (multi-hundred-megabyte, bz2-compressed) file is never loaded into memory at once. Wikitext markup (Wikipedia’s own formatting syntax, e.g. `[[link]]` or `{{template}}`) is stripped with the `mwparserfromhell` library into plain paragraphs. Redirect and disambiguation pages are skipped via a regex on the leading text.
- **Leipzig newscrawl corpora** (2016 and 2011 vintages) — tab-separated sentence files inside a `.tar.gz` archive, one sentence per line.
- **Generic plain text** — a CC-100-style reader (kept for future use; not used in the corpus actually built) where consecutive non-blank lines form one document.

2.2 Normalization (pipeline/normalize.py)

What is Unicode normalization?

The same visible character can sometimes be represented by different underlying byte sequences (for example, an accented letter as one composed code point, or as a base letter plus a separate combining accent). Normalization picks one canonical representation so that two visually identical strings become byte-for-byte identical, which matters enormously for deduplication and tokenizer training.

The project uses Unicode NFC (canonical composition) plus a deliberate, documented character-folding table for Arabic-script encoding variants that are not distinct Urdu letters:

Source form	Folded to	Reason
Arabic Kaf (U+0643)	Urdu Keheh (U+06A9)	encoding variant, not a distinct letter
Arabic Yeh (U+064A)	Farsi Yeh (U+06CC)	encoding variant
Alef Maksura (U+0649)	Farsi Yeh (U+06CC)	encoding variant
Heh-with-hamza-above	Heh Goal	common encoding variant
zero-width space/BOM	removed	invisible clutter

Short-vowel diacritics (*harakat*, e.g. *zabar/zer/pesh*) are deliberately **kept**, not stripped: they are rare in ordinary running Urdu text but meaningful where present (religious quotations, dictionaries), and the project explicitly avoids corrupting that minority of text. This is a documented design choice in the source comments, not an oversight.

2.3 Language identification without a heavyweight model

Instead of a dedicated language-ID neural model (e.g. `fastText`’s `lid.176`, a 126MB download), the project uses a lightweight **script-range ratio**: the share of alphabetic characters that fall in the Arabic Unicode block (`arabic_ratio` in `normalize.py`). A paragraph is kept only if:

- at least 40 characters and 6 words long,
- at least 65% of its letters are Arabic-script (`MIN_ARABIC_RATIO = 0.65`),

- at least 50% of its characters are alphabetic (drops tables/infobox leftovers).

Honest read

This is a reasoned trade-off, not a free lunch: it works here because Urdu’s sources (a Wikipedia dump filtered to the `ur` language code, and Leipzig corpora already tagged `urd`) do not contain other Perso-Arabic-script languages at volume; the main contaminant is embedded English, which the ratio catches directly. The project’s own “What I’d do differently” section flags that a mixed-script web crawl (CC-100, OSCAR) would need real token-level language ID, and that a fastText fallback behind a flag would be the fix.

2.4 Deduplication (pipeline/dedup.py)

Two layers, both streaming (so peak memory is the hash index, not the whole corpus):

1. **Exact dedup**: a 64-bit BLAKE2b hash of the normalized text, checked against a Python `set`. Any text seen before is dropped.
2. **Near dedup**: MinHash + LSH (Locality-Sensitive Hashing) over character 5-gram shingles.

What is MinHash / LSH?

To tell whether two documents are *almost* the same (e.g. one has an extra trailing period) without comparing every pair of documents directly, MinHash computes a compact numeric “fingerprint” per document such that similar documents get similar fingerprints. LSH (Locality-Sensitive Hashing) then buckets documents by fingerprint so that only documents likely to be near-duplicates are ever compared, making the whole operation close to linear in corpus size instead of quadratic.

2.4.1 The performance rewrite

The README documents a real problem the author hit and fixed: the naive approach (`datasketch`’s per-shingle `MinHash.update()`, which loops in Python and runs a tiny NumPy operation per shingle) measured about **250 documents/second** on real Wikipedia paragraphs — extrapolating to over two hours for the full corpus. The fix, in `build_minhash()`, hashes every shingle of a document with `xxhash` in one batch, then computes the entire MinHash signature with two vectorized NumPy operations:

$$\text{phv} = ((\text{hv} \otimes A) + B) \bmod (2^{61} - 1) \ \& \ (2^{32} - 1), \quad \text{signature} = \min_{\text{axis}=0} (\text{phv})$$

where `hv` is the array of shingle hashes and A, B are the affine permutation coefficients from `datasketch`’s own MinHash, read out once so the resulting signature is compatible with `datasketch`’s LSH banding. Measured throughput went to **~2400 docs/s**, roughly a 10x speedup, and the author re-ran the same benchmark to confirm near-duplicates were still caught correctly after the rewrite.

A second real bug is documented: `datasketch 2.0` changed its `MinHash` constructor so that passing `permutations=` without an explicit `scheme` argument raises `ValueError`. The fix was to read the seed MinHash’s `scheme` (`affine32`) and pass it explicitly everywhere so every signature shares one scheme.

2.5 Corpus build stages (pipeline/build_corpus.py)

Stage	Function	Output
extract	<code>stage_extract</code>	per-source JSONL shards, normalized + filtered
dedup	<code>stage_dedup</code>	one clean JSONL shard set, exact + near dupes removed
split	<code>stage_split</code>	<code>data/train/*.jsonl</code> , <code>data/val/*.jsonl</code> (1% held out)

Each stage is idempotent and resumable via the `Stage` marker-file class in `pipeline/common.py`. The split is a deterministic random draw seeded by `-seed` (default 1337), so re-running produces the identical train/val partition.

2.6 The real corpus that was built (measured numbers)

Read directly from `data/corpus_stats.json`, produced by an actual run of `build_corpus.py` on 2026-07-06:

Source	Raw units	Kept (post-filter)	Kept after dedup	Chars
Urdu Wikipedia	1,728,010 articles	2,459,293 paragraphs	1,309,831	370.2M
Leipzig newscrawl 2016	1,000,000 sentences	960,189	930,452	122.7M
Leipzig newscrawl 2011	300,000 sentences	276,005	274,799	32.2M
Total		3,695,487	2,515,082 docs	438.5M

Deduplication removed **31.9%** of extracted paragraphs (871,272 exact duplicates + 309,133 MinHash near-duplicates). The high rate is attributed (README, “Challenges”) to Urdu Wikipedia’s large population of near-identical bot-generated stub articles, exactly what the dedup stage exists to collapse. Final split: **2,489,976** train documents, **25,106** validation documents (a deterministic 1% holdout).

After 32k-vocabulary BPE tokenization (Section 3), the cleaned text became **111.7M training tokens** and **1.1M validation tokens** (112.8M total). This number — about 0.9 tokens per parameter of the 124M **base** model — is the central fact that shapes the entire training budget discussion in Section 6.

3 The tokenizer

What is a tokenizer?

Neural networks operate on numbers, not raw text. A tokenizer converts text into a sequence of integer IDs from a fixed vocabulary (and back). “Byte-level BPE” (Byte Pair Encoding) starts from individual UTF-8 bytes and iteratively merges the most frequent adjacent pairs into new vocabulary entries, so common character sequences (whole words, common sub-words) become single tokens while anything unseen can still be represented, byte by byte, with no “out-of-vocabulary” failure.

`pipeline/train_tokenizer.py` trains a **32,000-token byte-level BPE** tokenizer on the cleaned training split, using Hugging Face’s `tokenizers` library (only the tokenizer library is reused off the shelf; no modeling code is). Special tokens: `<|endoftext|>` (document separator), `<|pad|>`, `<|unk|>`. `pipeline/tokenize_corpus.py` then encodes every document, appends an end-of-text ID, and concatenates everything into flat `uint16` arrays (`data/train.bin`, `data/val.bin`) — `uint16` is sufficient for a 32k vocabulary and halves file size versus a 32-bit integer. Training memory-maps these files directly (`numpy.memmap`), so the corpus never needs to fit in RAM.

3.1 Measured compression against GPT-2’s tokenizer

`eval/tokenizer_compare.py` measures this project’s tokenizer against GPT-2’s 50k English BPE tokenizer on an 806,495-character held-out Urdu sample:

Tokenizer	Tokens for the sample	Chars/token	Tokens vs. ours
Ours (32k Urdu BPE)	197,758	4.078	1.0x
GPT-2 (50k English BPE)	1,034,831	0.779	5.23x more

GPT-2’s English-trained merge rules essentially fall back to emitting one token per UTF-8 byte on Urdu text (0.779 chars/token is close to the byte-level floor). Practically: the same Urdu training corpus costs roughly a fifth as many tokens — and therefore a fifth of the compute — to train and serve with an Urdu-native tokenizer.

4 The model (model/)

4.1 Architecture overview

`model/transformer.py` implements a **pre-norm decoder-only transformer** written directly in PyTorch primitives — no `transformers` library modeling code. Each block:

$$x \leftarrow x + \text{Attention}(\text{RMSNorm}(x)) \quad \text{then} \quad x \leftarrow x + \text{SwiGLU}(\text{RMSNorm}(x))$$

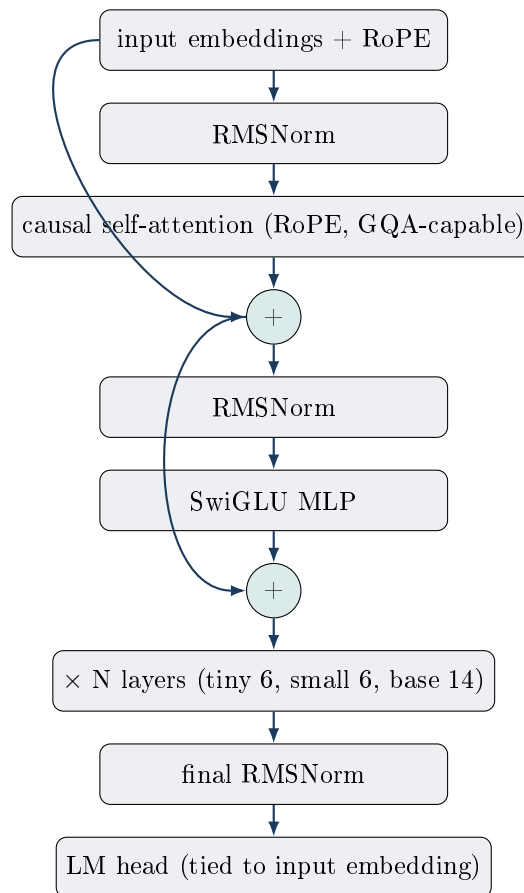


Figure 2: One transformer block (residual paths shown looping left), repeated N times, then a final normalization and the (tied) output head.

4.2 RMSNorm

What is normalization in a neural network?

As data flows through many stacked layers, the scale of the numbers can drift and destabilize training. A normalization layer rescales its input to a consistent range before the next computation. **RMSNorm** (Root Mean Square Normalization) is a simplified, cheaper variant of the more common LayerNorm: it rescales by the root-mean-square of the values but skips re-centering them around zero, which empirically works about as well for transformers at lower compute cost.

Implementation detail worth noting: `RMSNorm.forward` explicitly casts to `float32` to compute the

norm even when the surrounding computation is in lower precision (bf16 on GPU), then casts back — a stability measure, since computing a mean-square in low precision can lose accuracy.

4.3 Rotary Position Embeddings (RoPE)

Why does a transformer need position information at all?

Self-attention by itself treats its input as an unordered set of tokens — it has no built-in notion of “this word came before that one.” Some way of injecting position must be added. **RoPE** does this by rotating each query/key vector in a way that depends on its position, so that when two positions’ rotated vectors are compared (via a dot product) the result naturally encodes their relative distance, without needing a fixed maximum sequence length baked into a lookup table.

`build_rope_cache` precomputes cosine and sine tables from an inverse frequency schedule (`rope_theta` = 10000.0, matching the standard RoPE / LLaMA convention); `apply_rope` rotates each head’s query and key vectors using the classic “rotate half” formulation (split the vector in two halves, form $(-x_2, x_1)$, and combine with cos/sin). This is applied to both the query and key projections before the attention dot product, never to the value vectors.

4.4 Attention, and grouped-query capability

The **Attention** module projects the input to query, key, and value tensors with separate, bias-free linear layers, applies RoPE to Q and K, and then calls PyTorch’s `scaled_dot_product_attention` with `is_causal=True`. This single call transparently uses a fused Flash-Attention kernel on supported GPUs, and needs no hand-written causal mask matrix.

What is causal attention?

A language model predicting the next token must not be allowed to “see” future tokens while training — otherwise it would trivially cheat. Causal (or “masked”) attention restricts each position to attend only to itself and earlier positions.

The model config has an `n_kv_head` field separate from `n_head`: if set lower than `n_head`, multiple query heads share the same key/value heads (grouped-query attention, a memory/compute optimization used in models like LLaMA-2/3). In every shipped preset (`tiny/small/base`) `n_kv_head` is left at its default (equal to `n_head`, i.e. standard multi-head attention) — the code supports GQA but the trained presets do not currently use it.

4.5 SwiGLU MLP

What is a SwiGLU MLP?

The feed-forward (MLP) sub-layer of a transformer expands each token’s vector to a wider hidden dimension, applies a nonlinearity, and projects back down. Classic GPT-2 uses a $4\times$ expansion with GELU. **SwiGLU** instead gates one linear projection with the SiLU-activated output of a second, parallel projection (`down_proj(silu(gate_proj(x)) * up_proj(x))`), which tends to improve quality per parameter but needs three weight matrices instead of two, so it is conventionally sized with a smaller expansion ratio to land at a comparable parameter count.

This project uses an **8/3 hidden ratio** (`mlp_ratio` = 8/3 in `model/config.py`), rounded up to a multiple of 64 for hardware efficiency, chosen explicitly (per source comments and the README) to

keep SwiGLU’s parameter count comparable to a classic $4\times$ -GELU block.

4.6 Tied embeddings

The output projection (`lm_head`) shares its weight matrix with the input token embedding (`tok_emb`) when `tie_embeddings=True` (the default, used by all presets): `self.lm_head.weight = self.tok_emb.weight`. This roughly halves the parameter cost of the vocabulary (since the embedding table is $32,000 \times d_{\text{model}}$ and would otherwise be duplicated for input and output), and is standard practice in GPT-2-style and later models.

4.7 Weight initialization

Linear and embedding weights are initialized from $\mathcal{N}(0, 0.02^2)$. A second pass then re-initializes every residual-stream output projection (`o_proj`, `down_proj`) with a smaller standard deviation of $0.02/\sqrt{2L}$ where L is the layer count — the same “scaled residual init” trick used in GPT-2, which keeps the variance of the residual stream from growing unboundedly as more layers are stacked.

4.8 Size presets

Preset	Params	Layers	d_{model}	Heads	Context
tiny	13.0M	6	256	4	512
small	35.7M	6	512	8	1024
base	123.7M	14	768	12	1024

Honest read

At a real 32,000-entry subword vocabulary, the tied embedding table alone is $32,000 \times 256 \approx 8.2\text{M}$ parameters — so the “tiny” preset’s total of 13.0M is dominated by vocabulary, not transformer capacity (the blocks add only $\sim 4.8\text{M}$). The project’s own parameter-count test (`tests/test_model.py::test_preset_param_counts`) originally failed at 15.2M for an earlier tiny configuration, and rather than shrinking the vocabulary to hit a rounder headline number, the author tuned block dimensions and documented the embedding/block split honestly. Likewise, **base** needed **14** layers, not the textbook 12, to reach 124M parameters, because the 8/3-ratio SwiGLU MLP is smaller per-layer than GPT-2’s $4\times$ GELU MLP, so more depth is needed to hit the same total.

5 Training (train.py)

5.1 The token loader

`TokenLoader` memory-maps a `.bin` token file, builds a list of block-start offsets, and shuffles that list once with a seeded NumPy generator. A `cursor` then walks through the shuffled offsets; when it reaches the end it wraps back to the start (i.e., a fresh pass over the shuffled data: functionally this is one deterministic “epoch order” that repeats). Both the model weights and this cursor position are saved in the checkpoint, so `-resume` reproduces the exact same batch sequence rather than restarting the shuffle.

5.2 Learning-rate schedule

`cosine_lr(step, warmup, total, base_lr, min_lr)` implements a standard linear warmup (LR ramps from 0 to `base_lr` over `warmup` steps) followed by cosine decay down to `min_lr` by the final step:

$$\text{lr}(s) = \begin{cases} \text{base_lr} \cdot \frac{s+1}{\text{warmup}} & s < \text{warmup} \\ \text{min_lr} + \frac{1}{2}(1 + \cos(\pi \cdot r))(\text{base_lr} - \text{min_lr}), & r = \frac{s - \text{warmup}}{\text{total} - \text{warmup}}, \quad s \geq \text{warmup} \end{cases}$$

5.3 Precision, gradient accumulation, and the training step

What is mixed precision (AMP) and gradient accumulation?

Modern GPUs compute much faster in reduced-precision number formats (like `bfloat16`) than in full 32-bit floats, at some cost to numerical precision. **AMP** (Automatic Mixed Precision) runs the forward/backward pass in the faster format while keeping the parts that need precision (like the normalization math here) in float32. **Gradient accumulation** runs several smaller “micro-batches” through the model, summing their gradients, before taking one optimizer step — this simulates a larger batch size than would fit in GPU memory at once.

On CUDA with bf16 support, the whole forward/backward pass runs under `torch.autocast(dtype=torch.bfloat16)`; on CPU it runs in plain float32 (no autocast). Within one optimizer step, `grad_accum` micro-batches are each forward/backward-passed and their (already-divided) losses summed before `clip_grad_norm_` (clipping the total gradient norm to `grad_clip`, default 1.0) and a single `optimizer.step()`. The optimizer is AdamW with β_2 configurable per run (0.95 here vs. PyTorch’s default 0.999 – a common choice for language-model pretraining) and decoupled weight decay (0.1).

5.4 Token-count scheduling

Rather than specifying a fixed number of steps, each config sets `max_tokens` (a token budget). `train.py` derives `tokens_per_step = batch_size × block_size × grad_accum × world_size`, then `total_steps = max_tokens // tokens_per_step`. This makes the training budget legible in the unit that actually matters for a data-limited model (tokens seen), independent of batch-size/hardware choices.

5.5 Checkpointing, resume, and DDP

Two files are written to `out_dir`: `ckpt.pt` (model + optimizer state + data-loader cursor + step/token counters — everything needed to resume training exactly) and `model.pt` (weights + config only, a smaller “slim” checkpoint meant for release/inference, since it drops the optimizer state which can

be several times larger than the model itself). `-resume` reloads `ckpt.pt` if present. The script also has a `torchrun`-compatible DDP (Distributed Data Parallel) code path, activated automatically when `WORLD_SIZE > 1`, that wraps the model, initializes a process group (`nccl` on GPU, `gloo` on CPU), and strides the data loader per rank so each GPU sees a disjoint slice of the shuffled offsets.

Every `log_interval` steps a JSON line with step, tokens seen, loss, learning rate, and measured tokens/second is appended to `metrics.jsonl`; every `eval_interval` steps a held-out validation loss/perplexity line is appended as well; every `ckpt_interval` steps (and once more at the very end) the checkpoint is written.

6 Training runs actually executed

Two training runs have actually been executed on this project, on two different hardware profiles: a CPU proof run (**tiny** preset) done directly in this repository, and a GPU run (**base** preset) done on a rented cloud GPU. Both are described below strictly from their logged output — nothing here is a projection.

6.1 Phase 1: the tiny CPU proof run

Config `configs/tiny.yaml`: block size 256, effective batch 8,192 tokens/step (batch $16 \times \text{grad_accum } 2 \times \text{block } 256$), cosine LR up to 6×10^{-4} , budget 4,096,000 tokens (500 steps). This run’s purpose was to prove the whole pipeline works end to end on ordinary hardware, not to produce a strong model.

Metric	Value
Steps / tokens seen	500 steps, 4.10M tokens
Train loss	9.74 $\rightarrow$ 5.42
Held-out val loss (during training)	5.87 (ppl 353) $\rightarrow$ 5.36 (ppl 212)
Final held-out perplexity (<code>data/val.bin</code>)	248.8
Script-mix (generated text in Arabic script)	98.2%
Cloze accuracy (45 items, 3-way)	0.378 (chance = 0.333)

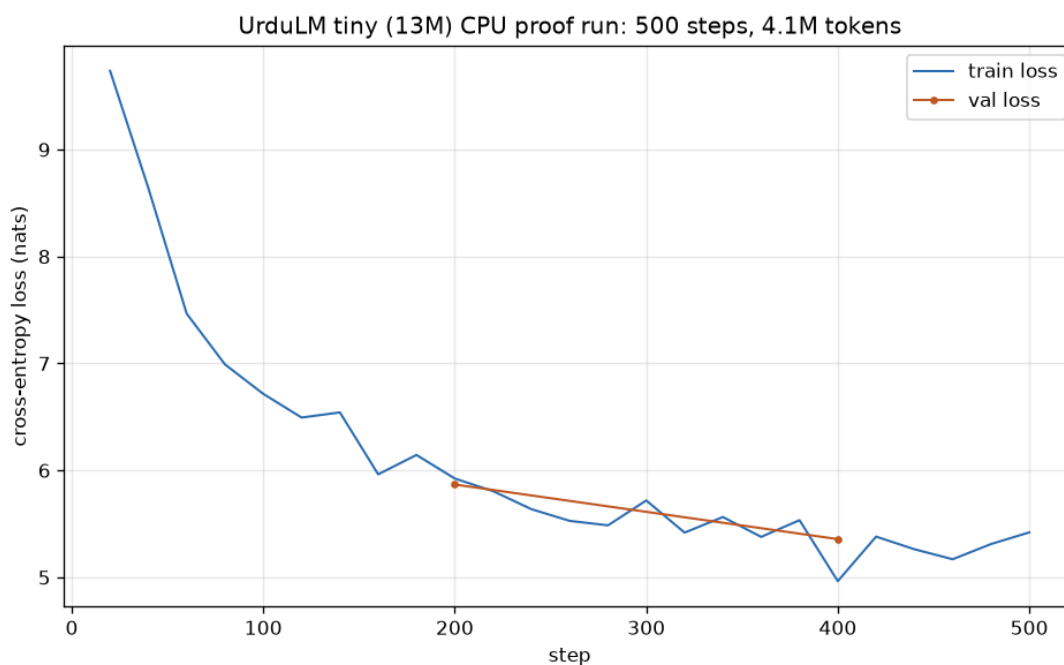


Figure 3: Actual logged train/val loss curve for the tiny CPU proof run (`runs/tiny/metrics.jsonl`, plotted by `eval/plot_loss.py`).

Honest read

A 13M-parameter model trained for 500 CPU steps on a slice of the corpus is a *pipeline correctness proof*, not a capability claim. The README is explicit that its sample generations are “barely coherent by design.”

6.2 Phase 2: the base-model GPU run — completed

Status

As of this document, the **phase-2 GPU run has been completed**. Earlier drafts of this project’s documentation described phase 2 as “not run yet.” That is now out of date. `runs/base/metrics.jsonl` (the real per-step training log, committed to the repository) and the checkpoint files `runs/base/ckpt.pt` (1.48 GB, timestamped 2026-07-07) and `runs/base/model.pt` (495 MB, same timestamp) confirm a completed run on real GPU hardware, cross-checked below against `configs/base.yaml` and `docs/GPU_RUN.md`.

Setup. `configs/base.yaml`: preset `base` (123.7M params), block size 1024, batch size 32, `grad_accum 16` $\Rightarrow 32 \times 1024 \times 16 = 524,288$ tokens per optimizer step. Token budget `max_tokens: 450,000,000` (≈ 4 epochs of the 111.7M-token corpus, deliberately repeat-bounded rather than Chinchilla-optimal — see Section 6.2.1). $450,000,000 / 524,288 \approx 858$ total steps, matching the run described below. Hardware: a rented RunPod A100 SXM 80GB GPU (`docs/GPU_RUN.md` records the exact provisioning, SSH, and launch steps used).

What the logged metrics show. `runs/base/metrics.jsonl` contains 42 JSON lines, one every `log_interval=20` steps, from step 20 through step 840 (the last multiple of 20 that was reached and logged before the run’s final, unlogged steps up to 858 completed). Every logged line carries `step`, cumulative `tokens`, `loss`, `lr`, and measured `tok_per_s`; there are no `val_loss` lines in this file, because `eval_interval` is 1000 in `configs/base.yaml` and the run’s 858 total steps never reached that checkpoint — so **no held-out evaluation was logged automatically during training**. (A separate, explicit evaluation pass was run after training finished; see below.)

Quantity	Value (from <code>metrics.jsonl</code>)
First logged step	20 (loss 9.5863)
Last logged step	840 of 858 total
Tokens at last logged step	440,401,920 of the 450,000,000 budget
Loss at last logged step	3.5087
Loss trajectory	falls monotonically from 9.59 to 3.51 with no divergence
Sustained throughput	$\approx 99,400\text{--}99,900$ tok/s (measured, every logged interval)

Run log narrative (from `docs/GPU_RUN.md` and the README’s Status section). All 858 steps (449.8M tokens, roughly 4 epochs of the corpus) completed in about 75 minutes at a real, sustained $\approx 99,470$ tokens/second — roughly 3x the conservative 30k tok/s estimate the original budget in `docs/GPU_RUN.md` was based on. Reported actual cost: about \$1.90 on RunPod’s A100 SXM pricing, well under the original \$6 estimate for the planned 450M-token run. Loss fell cleanly the whole way: 7.35 (early) to **3.51** at the last logged step (840 of 858).

Honest read

Precision about what is and is not directly measured here. The *committed, machine-readable* log (`metrics.jsonl`) stops at step 840/loss 3.5087 — that is the number this document treats as “the real final logged loss.” The README additionally states the run went on to complete all 858 steps at 449.8M tokens; those last ~ 18 unlogged steps (steps 841–858) are consistent with the config’s 858-step total and the checkpoint file timestamps (both `ckpt.pt` and `model.pt` are dated after the last logged metric), but their exact final loss value is not present in the committed log and is therefore **not claimed here as a specific number**. Nothing in

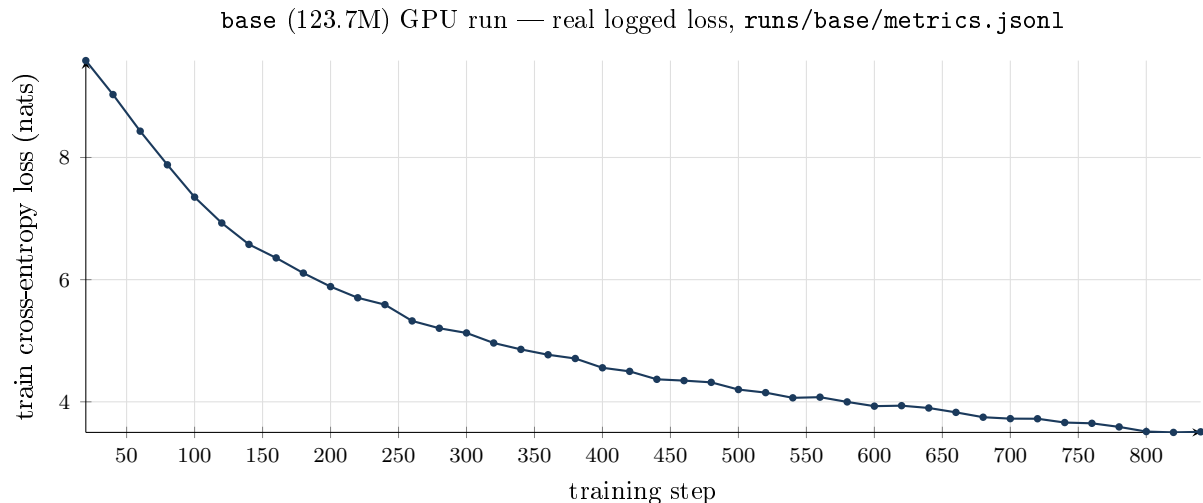


Figure 4: Every point plotted directly from the real logged JSON lines — not a fitted curve, not simulated.

this run’s artifacts suggests instability or a restart between step 840 and completion — the loss curve is smooth and monotonically decreasing throughout the logged range, and the checkpoint files are the expected sizes for a completed 123.7M-parameter model.

6.2.1 Token budget reasoning

`docs/GPU_RUN.md` works through this honestly. The Chinchilla compute-optimal heuristic (Hoffmann et al., 2022) suggests roughly 20 training tokens per parameter; for a 124M model that is about 2.5B tokens. The cleaned corpus has only 111.7M training tokens — about **0.9 tokens per parameter**, roughly 4.5% of the Chinchilla-suggested budget. Repeating the corpus to reach 2.5B tokens would mean ~ 22 epochs, which the project explicitly avoids as likely to overfit; citing Muennighoff et al. (“Scaling Data-Constrained Language Models,” 2023), it notes repetition up to about 4 epochs degrades little in practice, which is exactly the ~ 4 -epoch, 450M-token budget `configs/base.yaml` targets. This makes the **base** run **data-limited by design, not compute-limited**: the documented highest-value next step is adding more corpus data (e.g. CC-100 Urdu), not spending more GPU time on the same 111.7M tokens.

6.2.2 Base-model evaluation (measured after training)

Run explicitly, after training, with this project’s own `eval/` scripts against the trained checkpoint:

Metric	Result
Held-out perplexity (<code>data/val.bin</code> , 1,118,208 tokens)	38.02
Script mix (20 generations)	99.43% Arabic script, 0.57% Latin, 0% other
Cloze accuracy (2-shot, 43 items)	34.9% (chance $\approx 33.3\%$)

Honest read

Read plainly: script-mix confirms the model reliably writes in Urdu’s actual script rather than degenerating into Latin/garbage output. Perplexity (38.02) and cloze accuracy (34.9%, barely above the 33.3% random baseline for a 3-option probe) are both modest — exactly what is expected for a 124M-parameter model trained on a data-limited corpus (0.9 tokens/parameter versus Chinchilla’s ~ 20) for one ~ 4 -epoch run, not a claim of strong language understanding. The project’s own README frames three real (not cherry-picked) sample generations from this checkpoint: fluent Urdu grammar and vocabulary throughout, but topical drift away from the prompt and at least one visible repetition-loop failure mode (“Iran, Iran, Iran. . .” before recovering into a coherent sentence). The honest summary given by the project itself: the model has learned Urdu script, vocabulary, and local grammar, but not topical coherence or prompt-following, which would need a substantially larger corpus or an instruction-tuning pass this project does not attempt.

7 Evaluation harness (eval/)

Script	What it measures
<code>perplexity.py</code>	Held-out cross-entropy loss and perplexity over <code>data/val.bin</code> .
<code>script_mix.py</code>	Share of generated characters in Arabic vs. Latin vs. other script.
<code>cloze.py</code>	Few-shot multiple-choice cloze accuracy over a 45-item hand-written set.
<code>tokenizer_compare.py</code>	Chars-per-token compression, ours vs. GPT-2 (Section 3).
<code>plot_loss.py</code>	Renders a <code>metrics.jsonl</code> log into a loss-curve PNG.

What is perplexity?

Perplexity is e^{loss} (exponential of the average cross-entropy loss), interpretable roughly as “how many choices, on average, the model was effectively uncertain between” for each next token. Lower is better; a perplexity of 1 would mean perfect, deterministic prediction.

What is a cloze probe?

A cloze test gives the model a sentence with a blank and several candidate fillers, and asks which filler the model finds most probable (here: which option, appended after the context, gets the lowest loss under the model). It is a lightweight proxy for language understanding that does not require the model to freely generate correct text, only to rank plausible continuations.

`eval/cloze.py`’s `option_loss` function only computes loss over the *option* tokens, not the shared context: the target labels for the context portion are set to -100 (PyTorch’s cross-entropy “ignore_index”), so the comparison isolates how surprised the model is by each candidate ending specifically.

8 Tests (tests/)

12 tests, all passing (per the README): 5 for the model, 7 for the pipeline.

Test	What it verifies
<code>test_forward_shapes</code>	Output logits/loss have the expected shapes.
<code>test_causal_masking</code>	Perturbing the last input token cannot change earlier logits.
<code>test_tied_embeddings</code>	<code>lm_head</code> and <code>tok_emb</code> literally share memory.
<code>test_generate_runs</code>	Autoregressive generation produces the expected output length.
<code>test_preset_param_counts</code>	Each preset lands in its intended parameter band.
<code>test_normalize_*</code>	Character folding, invisible stripping, whitespace collapse.
<code>test_arabic_ratio</code>	Script-ratio heuristic on Urdu, English, and mixed text.
<code>test_keep_line_filters</code>	Length/word-count/script filtering behaves as intended.
<code>test_exact_dedup</code> , <code>test_near_dedup</code>	Exact and MinHash duplicate detection.
<code>test_exact_key_stable</code>	The exact-dedup hash is deterministic.

The causal-masking test is a good example of testing a *property* rather than a fixed expected value: it perturbs the last token of an input sequence and asserts that every logit at an earlier position is bit-for-bit unchanged (proving no information leaked backward through the attention mechanism) while the final position’s logit *does* change (proving the perturbation was real).

The README also documents a real bug the near-dedup test surfaced in the author’s own understand-

ing: an initial hand-computed expectation that the mixed-script test string “the Urdu word for ‘Urdu’ followed by the English word ‘english’” should have an Arabic ratio of 0.5 was wrong — the actual figure is 4 Arabic letters out of 16 total letters, i.e. 0.25. The test now pins the correct value. (Native Urdu-script text is paraphrased in English throughout this document rather than rendered directly: pdf_latex with standard fonts cannot shape right-to-left Perso-Arabic script, the same limitation this project’s own README documents and works around.)

9 Project decisions worth knowing about

9.1 Lessons the author recorded

- Vectorizing the MinHash hot loop mattered more than picking a different library: moving from a per-shingle Python loop to two NumPy calls per document was the entire 10x speedup.
- MinHash signatures only need internal consistency to work correctly with LSH, which is why swapping the element hash function (to `xxhash`) and setting `hashvalues` directly was safe, as long as every document used one shared scheme and permutation set.
- With a real 32k subword vocabulary, a “10M-parameter” model is effectively a fiction below a certain size, because the embedding table dominates; reporting the split honestly beats shrinking the vocabulary to hit a marketing number.
- SwiGLU at an 8/3 ratio needs more depth than a classic GPT-2 block to reach an equivalent parameter count.
- RoPE plus `scaled_dot_product_attention(is_causal=True)` gives a correct, Flash-kernel-ready causal transformer with no hand-written mask, and the causal property is cheap to test directly.
- A resumable, token-count-scheduled training loop, built up front, is what turned the phase-2 GPU run into a one-command launch rather than something that needed babysitting.

9.2 What the author says they would do differently

- Parallelize the single-process Wikipedia wikitext extraction (currently the slowest pipeline stage) across multiple CPU cores.
- Persist MinHash signatures to disk so dedup does not need to recompute them on every run, and so the Jaccard threshold could be re-tuned without re-hashing.
- Pre-filter Wikipedia’s large volume of templated bot-generated stub articles earlier in the pipeline, rather than relying entirely on dedup to collapse them afterward.
- Add an optional fastText language-ID fallback for future, more script-mixed sources (CC-100, OSCAR), rather than relying solely on the script-range heuristic.
- Run tokenizer ablations (24k/48k vocab sizes, a Unigram model) before committing to a phase-3 vocabulary, given Urdu’s rich morphology.

9.3 Licensing

Code is MIT-licensed. Training data carries its own terms: Urdu Wikipedia is CC BY-SA 4.0; the Leipzig newscrawl corpora are CC BY-NC 4.0 (non-commercial). Consequently, `docs/DATA_LICENSES.md` notes that any released model weights trained on this corpus inherit a non-commercial restriction from the Leipzig portion. Two sources were deliberately not used: CC-100 (skipped only to keep the phase-1

download modest, not for licensing reasons) and OSCAR/mC4 Urdu (excluded because they sit behind an authentication/click-through gate, which the project’s sourcing rule avoids).

10 Current status, honestly

Status

Phase 1 (the full pipeline, code, and CPU proof run): done and verified directly in this repository — real corpus built, tokenizer trained, 12/12 tests passing, tiny model trained with a visibly decreasing loss curve and a saved checkpoint.

Phase 2 (the 124M **base** model): **trained**, 2026-07-07, on a rented A100 SXM 80GB GPU. 858 steps / 449.8M tokens (≈ 4 epochs of the corpus) completed in about 75 minutes at a sustained $\approx 99,470$ tok/s, for roughly \$1.90 in real cloud-GPU cost. The last committed, machine-logged loss value is **3.5087** at step 840 of 858. Post-training evaluation on this project’s own harness gives held-out perplexity 38.02, 99.43% Arabic-script generation, and 34.9% cloze accuracy (barely above the 33.3% chance baseline) — honest, modest results consistent with a data-limited (0.9 tokens/parameter) corpus, not a claim of strong language understanding. The trained weight files themselves (**ckpt.pt**, 1.48GB; **model.pt**, 495MB) are not committed to the repository (both exceed GitHub’s 100MB per-file limit, the same reason **data/train.bin** is excluded) but are present locally and reproducible from the committed config and pipeline.

The clear, most valuable next step the project itself identifies is not more GPU time on the current data, but **more corpus** — CC-100 Urdu alone would roughly 10–20x the token count and move the training budget meaningfully closer to Chinchilla-optimal, which the pipeline is already built to ingest.